Assignment 2: Generative Models of Images

Generative AI & LLM — Fall 2026, ITU

Fahad Khalid

November 2026

Code repository	https://github.com/Fahad00888/genai-assignment2 (private, shared with HajraZawwar)
Model checkpoint	Google Drive: model.pt (10k-step model)
Experiment tracking	https://forge.coreweave.com/wandb/fahad00888-itu-lahore/DDPM_AFHQ

Contents

1 Convolutional Neural Networks **2**

2 Encoder-only Transformers **3**

3 Generative Adversarial Network (GAN) **3**

4 Variational Autoencoders **4**

5 Understanding Diffusion Models **5**

6 Programming: Diffusion Models **6**

1 Convolutional Neural Networks

1.1.a Padding

With $m = h - \lfloor K_h/2 \rfloor + (i-1)$ and $i \in \{1, \ldots, K_h\}$, the extreme row indices are

$$m_{\min} = 1 - \left\lfloor \tfrac{K_h}{2} \right\rfloor \quad (h = 1,\ i = 1), \qquad m_{\max} = N_h + K_h - 1 - \left\lfloor \tfrac{K_h}{2} \right\rfloor \quad (h = N_h,\ i = K_h).$$

Rows $m < 1$ and $m > N_h$ are hallucinated, so the number of missing rows above is $1 - m_{\min}$ and below is $m_{\max} - N_h = K_h - 1 - \lfloor K_h/2 \rfloor = \lfloor (K_h - 1)/2 \rfloor$. Columns behave identically with K_w:

$$\boxed{p_l = \left\lfloor \frac{K_w}{2} \right\rfloor, \qquad p_r = \left\lfloor \frac{K_w - 1}{2} \right\rfloor, \qquad p_t = \left\lfloor \frac{K_h}{2} \right\rfloor, \qquad p_b = \left\lfloor \frac{K_h - 1}{2} \right\rfloor}$$

Sanity checks: $K = 3$ gives 1 on both sides; an even kernel $K = 4$ gives 2 before and 1 after; in all cases $p_t + p_b = K_h - 1$ and $p_l + p_r = K_w - 1$.

1.1.b Convolution on the explicitly padded input

Define the padded tensor $\mathbf{x}' \in \mathbb{R}^{C \times (N_h + p_t + p_b) \times (N_w + p_l + p_r)}$ by

$$x'_{c,a,b} = \begin{cases} x_{c,\, a - p_t,\, b - p_l} & \text{if } 1 \le a - p_t \le N_h \text{ and } 1 \le b - p_l \le N_w, \\ 0 & \text{otherwise.} \end{cases}$$

The new layer is

$$y'_{1,h,w} = \theta_0 + \sum_{c=1}^{C} \sum_{i=1}^{K_h} \sum_{j=1}^{K_w} \theta_{c,i,j}\, x'_{c,m',n'}, \tag{1}$$

$$\text{where } m' = h + (i-1) \text{ and } n' = w + (j-1), \tag{2}$$

$$\forall h \in \{1, \ldots, N_h\},\ w \in \{1, \ldots, N_w\}. \tag{3}$$

Same output: $m' = m + p_t$ and $n' = n + p_l$, so each product reads $x'_{c,m+p_t,n+p_l} = x_{c,m,n}$ for real pixels and 0 exactly where the original formulation hallucinated a zero. *Valid indices only:* m' ranges over $1, \ldots, N_h + K_h - 1 = N_h + p_t + p_b$ and n' over $1, \ldots, N_w + p_l + p_r$, i.e. exactly the extent of $\mathbf{x}'$, and $\mathbf{y}'$ keeps the shape $1 \times N_h \times N_w$.

1.1.c.i Role of skip connections in U-Net

Skip connections take the encoder's feature map at each resolution and concatenate it to the decoder's feature map at the same resolution. Every downsampling step in the encoder discards spatial detail, so the bottleneck only carries a coarse, semantic summary (*what* is in the image). Without skips the decoder would have to invent the exact boundaries from that summary and the masks would be blurry and misplaced. The skips hand back the high-resolution information (edges, precise positions) so the decoder can combine *what* (from the deep path) with *where* (from the skip), which is exactly what per-pixel segmentation needs. They also give gradients short paths to early layers, which eases training.

1.1.c.ii Spatial resolution through the network

In the encoder each level *halves* the height and width (2×2 max-pooling in the original U-Net, a stride-2 convolution in our implementation) while the number of channels typically *doubles*; the bottleneck has the smallest spatial size and the most channels. The decoder mirrors this: every level *doubles* the height and width (up-convolution / upsampling) and halves the channels until the input resolution is restored. In our diffusion U-Net a 32×32 input goes $32 \to 16 \to 8 \to 4$ and back $4 \to 8 \to 16 \to 32$. (In the original paper the unpadded 3×3 convolutions also shave a border off each map, so a 572×572 input gives a 388×388 output.)

2 Encoder-only Transformers

2.1 Graphical model of a decoder-only Transformer

A decoder-only Transformer factorises the joint with the chain rule, $p(w_1, \ldots, w_N) = \prod_{n=1}^{N} p(w_n \mid w_1, \ldots, w_{n-1})$, and makes *no* conditional-independence assumption: through causal self-attention every word can depend on every earlier word. The directed graph therefore has an edge from *every* earlier node to every later node (a fully connected DAG in left-to-right order). Drawn for $N = 5$:

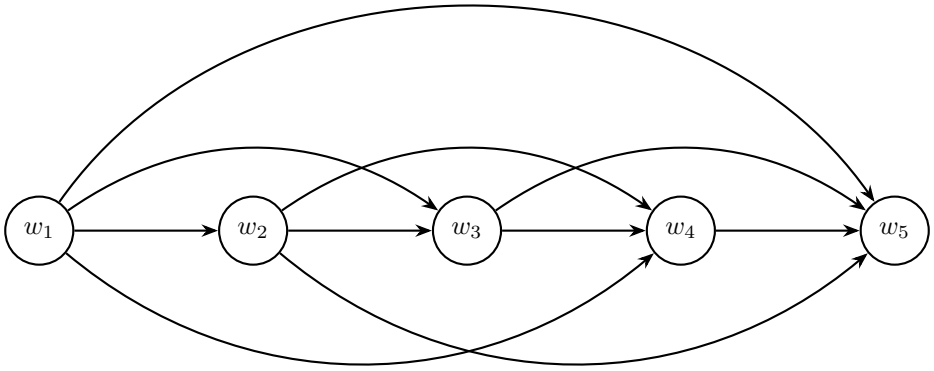


Figure 1: Directed graphical model of $p(w_1, \ldots, w_5)$ under a decoder-only Transformer: node w_n has parents $w_1, \ldots, w_{n-1}$. For general N the same pattern continues (all $\binom{N}{2}$ forward edges).

2.2 Encoder-only vs. decoder-only sequence distributions

Encoder-only (with the given output layer). Normalising the score vector gives a per-token distribution over tags, $p(y_n = v \mid w_{1:N}) = s_{n,v} / \sum_{v'} s_{n,v'}$ (a softmax of $\mathbf{W}\mathbf{h}_n + \mathbf{b}$). Because $\mathbf{h}_n$ comes from bidirectional attention, each tag sees the *whole* sentence, but the tags are predicted independently of one another given the input:

$$p(y_1, \ldots, y_N \mid w_{1:N}) = \prod_{n=1}^{N} p(y_n \mid w_{1:N}).$$

So this is a *conditional* distribution over a label sequence of the *same length* as the input, with no direct dependencies between output labels; it does not define a distribution over the words themselves and cannot generate text.

Decoder-only. It defines the full *joint* distribution over the token sequence itself, $p(w_{1:N}) = \prod_n p(w_n \mid w_{<n})$, with no independence assumptions, but each position only uses *left* context. Because it is a proper generative model, it can sample sequences of arbitrary length token by token and can score any sequence.

Best suited for. Encoder-only: tasks where the input is fully given and we need a label per token or per sequence: POS tagging, named-entity recognition, sentence classification, extractive QA, retrieval embeddings. Decoder-only: tasks that require producing text: language modelling, completion, dialogue, summarisation or translation phrased as continuation, and scoring the likelihood of sequences.

3 Generative Adversarial Network (GAN)

Throughout, $\odot$ is element-wise multiplication and $m_{ij} = 1$ marks a pixel to be filled in.

3.1.a Blank (a): composed output

$$\mathbf{x}' = \mathbf{m} \odot g_\theta(\mathbf{x}, \mathbf{m}) + (\mathbf{1} - \mathbf{m}) \odot \mathbf{x}$$

The model only decides the masked pixels; every intact pixel is copied from the input.

3.1.b Blank (b): masked squared error

$$\ell_{mse}(\theta) = \big\|\mathbf{m} \odot \big(g_\theta(\mathbf{x}, \mathbf{m}) - \mathbf{x}\big)\big\|_2^2 = \sum_{i=1}^{N_h} \sum_{j=1}^{N_w} m_{ij} \left(g_\theta(\mathbf{x}, \mathbf{m})_{ij} - x_{ij}\right)^2$$

Unmasked pixels are multiplied by $m_{ij} = 0$ and do not contribute (equivalently $\ell_{mse} = \|\mathbf{x}' - \mathbf{x}\|_2^2$, since $\mathbf{x}'$ equals $\mathbf{x}$ outside the mask). Dividing by $\sum_{ij} m_{ij}$ gives a per-pixel mean if masks vary in size.

3.1.c Blank (c): GAN-style objective

$$\ell_{gan}(\phi, \theta) = \log d_\phi(\mathbf{x}) + \log\Big(1 - d_\phi\big(\mathbf{m} \odot g_\theta(\mathbf{x}, \mathbf{m}) + (\mathbf{1} - \mathbf{m}) \odot \mathbf{x}\big)\Big), \qquad \min_\theta \max_\phi \ \ell_{gan}(\phi, \theta).$$

The discriminator ascends ℓ_{gan} (high probability for the real image, low for the inpainted one); the inpainting network descends it, which matches the update $\theta \leftarrow \theta - \eta \nabla_\theta \ell$ in the pseudo-code. Only the second term depends on θ.

3.1.d Disadvantage of training with ℓ_{mse} only

A masked region usually has many plausible fillings (different fur patterns, ear positions, ...). Squared error is minimised by the *average* of these possibilities, so an MSE-only model learns to output their mean: smooth, blurry patches without realistic texture, which stand out against the sharp surrounding pixels. The adversarial term instead penalises outputs that look unlike real images, pushing the model to commit to one sharp, realistic completion; combining both keeps the completion close to the truth (MSE) while staying on the manifold of natural images (GAN).

4 Variational Autoencoders

4.1 Which parameters receive gradients from $\widehat{\mathcal{R}}$?

$\widehat{\mathcal{R}} = \log p_\theta(\mathbf{x} \mid z) = -\frac{(\mathbf{x} - w_3 z - b_3)^2}{2\sigma^2} + \text{const.}$

(a) **Black-box sampling:** only $\boldsymbol{w_3}, \boldsymbol{b_3}$. The node "$z \sim q_\phi(z \mid \mathbf{x})$" outputs a number with no differentiable path back to μ_ϕ or σ_ϕ, so $\partial\widehat{\mathcal{R}}/\partial w_1 = \partial\widehat{\mathcal{R}}/\partial b_1 = \partial\widehat{\mathcal{R}}/\partial w_2 = \partial\widehat{\mathcal{R}}/\partial b_2 = 0$ in this graph, while $\widehat{\mathcal{R}}$ depends on w_3, b_3 directly.

(b) **Reparameterisation: all six, $\boldsymbol{w_1}, \boldsymbol{b_1}, \boldsymbol{w_2}, \boldsymbol{b_2}, \boldsymbol{w_3}, \boldsymbol{b_3}$.** Now $z = (w_1\mathbf{x} + b_1) + \exp\big(\frac{w_2\mathbf{x} + b_2}{2}\big)\,\epsilon$ is a deterministic, differentiable function of ϕ (randomness sits in ϵ, which has no parameters), so the chain rule $\frac{\partial\widehat{\mathcal{R}}}{\partial\phi} = \frac{\partial\widehat{\mathcal{R}}}{\partial z}\frac{\partial z}{\partial\phi}$ is non-zero for w_1, b_1 (through μ_ϕ) and w_2, b_2 (through σ_ϕ), and w_3, b_3 still get gradients directly.

4.2 Batch loss for video inputs

Each video has $D = 3FHW$ pixels. Per-sample reconstruction error (MSE normalised by the number of pixels):

$$\mathcal{L}_i^{rec} = \frac{1}{3FHW} \sum_{c=1}^{3} \sum_{f=1}^{F} \sum_{h=1}^{H} \sum_{w=1}^{W} \left(x_{i,cfhw} - \tilde{x}_{i,cfhw}\right)^2.$$

KL to the prior, using the hint with $\mu_0 = \mu_\phi(x_i)$, $\sigma_0^2 = \sigma_\phi^2(x_i)$, $\mu_1 = 0$, $\sigma_1^2 = 1$:

$$D_{KL}\big(\mathcal{N}(\mu_\phi(x_i), \sigma_\phi^2(x_i)) \,\|\, \mathcal{N}(0,1)\big) = \frac{1}{2}\Big(\log \frac{1}{\sigma_\phi^2(x_i)} + \sigma_\phi^2(x_i) + \mu_\phi(x_i)^2 - 1\Big).$$

Summing per-sample losses, weighting the KL by β and multiplying by $1/N$ (batch size N):

$$\boxed{\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N}\left[\frac{1}{3FHW}\big\|x_i - \tilde{x}_i\big\|_2^2 + \frac{\beta}{2}\Big(\sigma_\phi^2(x_i) + \mu_\phi(x_i)^2 - 1 - \log\sigma_\phi^2(x_i)\Big)\right]}$$

where $\|\cdot\|_2^2$ sums over all $3FHW$ entries.

4.3 Increasing β

The distribution $q_\phi(z \mid x)$ approaches $\mathcal{N}(0, I)$ but reconstructions become worse. A larger β makes it more costly for the encoder to move $q_\phi(z \mid x)$ away from the prior, so the codes of different inputs overlap more and carry less information about x; the decoder then has less to work with and reconstructions degrade (the classic rate–distortion trade-off).

5 Understanding Diffusion Models

5.1 ELBO surgery

Step 1: expand both chains. Substituting Eq. (5) and Eq. (6) into Eq. (7) and taking logs,

$$\log \frac{p_\theta(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T} \mid \mathbf{x}_0)} = \log p(\mathbf{x}_T) + \sum_{t=1}^{T} \log p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t) - \sum_{t=1}^{T} \log q(\mathbf{x}_t \mid \mathbf{x}_{t-1}).$$

Step 2: re-index the reverse terms so that each one lines up with a forward term of the same $\mathbf{x}_t$. Peel off $t = 1$ and shift $t \to t+1$ in the rest:

$$\sum_{t=1}^{T} \log p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t) = \log p_\theta(\mathbf{x}_0 \mid \mathbf{x}_1) + \sum_{t=1}^{T-1} \log p_\theta(\mathbf{x}_t \mid \mathbf{x}_{t+1}).$$

Split the last forward term off: $\sum_{t=1}^{T} \log q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) = \sum_{t=1}^{T-1} \log q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) + \log q(\mathbf{x}_T \mid \mathbf{x}_{T-1})$.

Step 3: regroup into $R + \sum \mathcal{L}_t + L_T$.

$$\log \frac{p_\theta(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T} \mid \mathbf{x}_0)} = \underbrace{\log p_\theta(\mathbf{x}_0 \mid \mathbf{x}_1)}_{R} - \sum_{t=1}^{T-1} \log \frac{q(\mathbf{x}_t \mid \mathbf{x}_{t-1})}{p_\theta(\mathbf{x}_t \mid \mathbf{x}_{t+1})} + \underbrace{\log \frac{p(\mathbf{x}_T)}{q(\mathbf{x}_T \mid \mathbf{x}_{T-1})}}_{L_T}.$$

Step 4: take $\mathbb{E}_{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}$ and marginalise. Each term depends on only a few variables, so by $\mathbb{E}_{q(a,b,c)}[f(b,c)] = \mathbb{E}_{q(b,c)}[f(b,c)]$ the remaining variables integrate out of $q(\mathbf{x}_{1:T} \mid \mathbf{x}_0)$:

- R depends on $(\mathbf{x}_0, \mathbf{x}_1)$ only $\Rightarrow \mathbb{E}_{q(\mathbf{x}_1|\mathbf{x}_0)}[\log p_\theta(\mathbf{x}_0 \mid \mathbf{x}_1)]$;
- the t-th summand depends on $(\mathbf{x}_{t-1}, \mathbf{x}_t, \mathbf{x}_{t+1})$ only $\Rightarrow \mathbb{E}_{q(\mathbf{x}_{t-1},\mathbf{x}_t,\mathbf{x}_{t+1}|\mathbf{x}_0)}[\,\cdot\,]$. (The Markov structure of the forward chain is what guarantees this triple marginal is all we need: given $\mathbf{x}_t$, everything before and after is irrelevant to the summand.)
- L_T depends on $(\mathbf{x}_{T-1}, \mathbf{x}_T)$ only.

Step 5: identify the constant. $\mathbb{E}[L_T] = -\mathbb{E}_{q(\mathbf{x}_{T-1}|\mathbf{x}_0)}\big[D_{KL}(q(\mathbf{x}_T \mid \mathbf{x}_{T-1}) \,\|\, p(\mathbf{x}_T))\big]$ involves only the fixed prior $p(\mathbf{x}_T) = \mathcal{N}(\mathbf{0}, \mathbf{I})$ and the fixed forward process (no learnable parameters), so it does not depend on θ; call it C. Putting everything together:

$$\log p(\mathbf{x}) \;\geq\; \mathbb{E}_{q(\mathbf{x}_1|\mathbf{x}_0)}\big[\log p_\theta(\mathbf{x}_0 \mid \mathbf{x}_1)\big] - \sum_{t=1}^{T-1} \underbrace{\mathbb{E}_{q(\mathbf{x}_{t-1},\mathbf{x}_t,\mathbf{x}_{t+1}|\mathbf{x}_0)}\left[\log \frac{q(\mathbf{x}_t \mid \mathbf{x}_{t-1})}{p_\theta(\mathbf{x}_t \mid \mathbf{x}_{t+1})}\right]}_{\mathcal{L}_t} + C. \qquad \blacksquare$$

5.2 Effect of $\mathcal{L}_t$ on the reverse process

$\mathcal{L}_t$ enters the ELBO with a minus sign, so maximising the ELBO *minimises* $\mathcal{L}_t$. Its numerator $q(\mathbf{x}_t \mid \mathbf{x}_{t-1})$ is fixed, so the only θ-dependent part is $-\mathbb{E}_q[\log p_\theta(\mathbf{x}_t \mid \mathbf{x}_{t+1})]$: a cross-entropy that rewards the reverse step for placing high probability on the $\mathbf{x}_t$ values the forward process actually produced before it reached $\mathbf{x}_{t+1}$. In words, $\mathcal{L}_t$ forces each denoising step $p_\theta(\mathbf{x}_t \mid \mathbf{x}_{t+1})$ to *undo one step of noising*, i.e. to be consistent with the forward chain at the same time step. $\mathcal{L}_t$ grows without bound when p_θ puts almost no mass where the forward samples are; the ELBO term $-\mathcal{L}_t$ is maximised (and $\mathcal{L}_t$ minimised) when the reverse step matches the forward chain's own reverse conditional, $p_\theta(\mathbf{x}_t \mid \mathbf{x}_{t+1}) = q(\mathbf{x}_t \mid \mathbf{x}_{t+1}, \mathbf{x}_0)$, at which point what remains of $\mathcal{L}_t$ no longer depends on θ.

5.3 Sampling $\mathbf{x}_\tau$ with the one-step encoder

Use Eq. (9) as a recipe and walk the chain from $\mathbf{x}_0$:

1. Start with the data point $\mathbf{x}_0$.
2. For $t = 1, 2, \ldots, \tau$: draw $\boldsymbol{\epsilon}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and set $\mathbf{x}_t = \sqrt{\alpha_t}\,\mathbf{x}_{t-1} + \sqrt{1-\alpha_t}\,\boldsymbol{\epsilon}_t$ (reparameterised sample of $\mathcal{N}(\sqrt{\alpha_t}\mathbf{x}_{t-1}, (1-\alpha_t)\mathbf{I})$).
3. Return $\mathbf{x}_\tau$.

Each iteration needs the previous one, so this costs τ Gaussian draws and updates: $\boldsymbol{O(\tau)}$ (times $O(d)$ per step for a d-pixel image).

5.4 Closed form of $q(\mathbf{x}_t \mid \mathbf{x}_0)$ (proof by induction)

Claim: $\mathbf{x}_t = \sqrt{\bar\alpha_t}\,\mathbf{x}_0 + \sqrt{1-\bar\alpha_t}\,\boldsymbol{\epsilon}$ with $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, i.e. $q(\mathbf{x}_t \mid \mathbf{x}_0) = \mathcal{N}(\sqrt{\bar\alpha_t}\mathbf{x}_0, (1-\bar\alpha_t)\mathbf{I})$.

Base case $t = 1$: By Eq. (10) applied to Eq. (9), $\mathbf{x}_1 = \sqrt{\alpha_1}\mathbf{x}_0 + \sqrt{1-\alpha_1}\boldsymbol{\epsilon}_1$, and $\bar\alpha_1 = \alpha_1$. ✓

Inductive step: assume $\mathbf{x}_{t-1} = \sqrt{\bar\alpha_{t-1}}\mathbf{x}_0 + \sqrt{1-\bar\alpha_{t-1}}\,\boldsymbol{\epsilon}'$. One more forward step with fresh, independent noise $\boldsymbol{\epsilon}_t$:

$$\begin{aligned}\mathbf{x}_t &= \sqrt{\alpha_t}\,\mathbf{x}_{t-1} + \sqrt{1-\alpha_t}\,\boldsymbol{\epsilon}_t \\ &= \sqrt{\alpha_t\bar\alpha_{t-1}}\,\mathbf{x}_0 + \underbrace{\sqrt{\alpha_t(1-\bar\alpha_{t-1})}\,\boldsymbol{\epsilon}' + \sqrt{1-\alpha_t}\,\boldsymbol{\epsilon}_t}_{\text{sum of two independent zero-mean Gaussians}}\ .\end{aligned}$$

The sum of independent Gaussians is Gaussian with the variances added: $\alpha_t(1-\bar\alpha_{t-1}) + (1-\alpha_t) = 1 - \alpha_t\bar\alpha_{t-1} = 1 - \bar\alpha_t$, using $\bar\alpha_t = \alpha_t\bar\alpha_{t-1}$. Hence the noise part equals $\sqrt{1-\bar\alpha_t}\,\boldsymbol{\epsilon}$ in distribution, with $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, and

$$\mathbf{x}_t = \sqrt{\bar\alpha_t}\,\mathbf{x}_0 + \sqrt{1-\bar\alpha_t}\,\boldsymbol{\epsilon} \quad\Longleftrightarrow\quad q(\mathbf{x}_t \mid \mathbf{x}_0) = \mathcal{N}\big(\mathbf{x}_t; \sqrt{\bar\alpha_t}\,\mathbf{x}_0, (1-\bar\alpha_t)\mathbf{I}\big). \qquad \blacksquare$$

5.5 Sampling $\mathbf{x}_\tau$ in closed form

1. Precompute $\bar\alpha_t = \prod_{i \le t} \alpha_i$ for all t once (a cumulative product, done once before training; our code stores it as a buffer).
2. Draw a single $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.
3. Return $\mathbf{x}_\tau = \sqrt{\bar\alpha_\tau}\,\mathbf{x}_0 + \sqrt{1-\bar\alpha_\tau}\,\boldsymbol{\epsilon}$.

With $\bar\alpha$ precomputed this is a table lookup plus one noise draw: $\boldsymbol{O(1)}$ in τ (vs. $O(\tau)$ in 5.3). Even computing $\bar\alpha_\tau$ on the fly costs only τ scalar multiplications and still a single image-sized noise draw. This is what lets training pick a random t per image in constant time.

6 Programming: Diffusion Models

Setup and implementation notes

- **Data and model.** AFHQ cat training split (5,153 images, resized to 35 and randomly cropped/flipped to 32×32, scaled to $[-1, 1]$); $T = 50$ with the cosine schedule ($s = 0.008$);

U-Net with base width 16 and multipliers $(1, 2, 4, 8)$; batch 32 with 2-step gradient accumulation; Adam, learning rate 10^{-3}; L1 noise-prediction loss.

- `diffusion.py`. All schedule quantities (α_t, $\bar{\alpha}_t$, $\bar{\alpha}_{t-1}$ with $\bar{\alpha}_{-1} = 1$, the forward scales $\sqrt{\bar{\alpha}_t}$, $\sqrt{1 - \bar{\alpha}_t}$, and the posterior coefficients and standard deviation of Eq. 18) are computed once and stored with `register_buffer`, so they move with the model to the device and are saved in checkpoints without being trained. At the final step the posterior variance is exactly 0 and the posterior mean reduces to the clamped $\hat{\mathbf{x}}_0$ (verified numerically).
- **Hardware.** All runs used a MacBook Pro (Apple M5, PyTorch MPS backend), *not* Colab. The full 10k-step run took 31.5 minutes.
- **Fixes to the provided code** (each is a separate commit in the repository):
  1. `main.py --visualize` passed the argument dictionary positionally; it is now unpacked with `**`.
  2. clean-fid defaults to a CUDA device; FID is now computed on the CPU so it runs on Apple Silicon.
  3. The FID reference folder was `data/val`, which mixes cat, dog and wild images; it now uses `data/val/cat` (500 real cats) so generated cats are compared with real cats.
  4. The backward visualisation started from the forward image *after* it had been clamped and rescaled to $[0, 1]$ for display, instead of the actual noisy sample in model space $[-1, 1]$; it now starts from the raw $\mathbf{x}_{49}$.
  5. On the MPS backend, images logged to wandb came out completely black (the float→uint8 conversion happened on the GPU); tensors are now moved to the CPU before logging, and the forward/backward figures are also written to disk.

Run	Question	Settings	Time	Final loss
`run_1k`	6.1	1,000 steps, sample every 100	5 min 23 s	0.182
`run_1k_fid`	6.2	1,000 steps, sample + FID every 100	19 min 24 s	0.184
`run_10k`	6.3–6.5	10,000 steps, sample every 1,000	31 min 30 s	0.159

Table 1: Training runs (all on Apple M5 / MPS). Full log in `experiments.md`.

6.1 Training loss

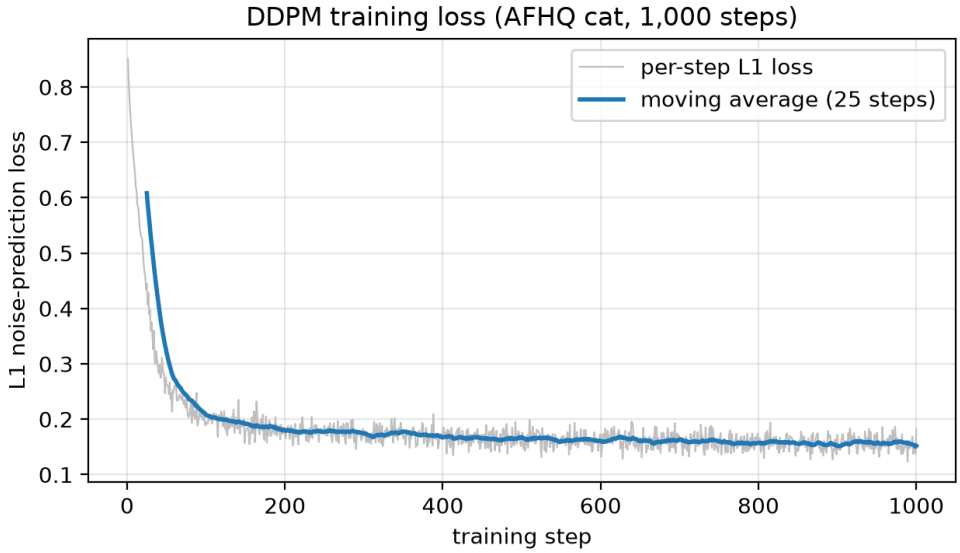


Figure 2: Per-step L1 loss and its moving average over 1,000 training steps.

The loss falls steeply in the first ∼200 steps (from 0.85 at the first step to about 0.2) as the network learns the easy part of the task: predicting the noise at large t, where $\mathbf{x}_t$ is almost pure noise. After that it flattens, with the 25-step moving average ending at 0.15 while individual steps keep fluctuating noticeably around it. The fluctuations are expected: each step draws a different random timestep per image, and the difficulty varies strongly with t (at small t the added noise is tiny and hard to separate from image detail), so the batch loss is a noisy estimate. Continuing to 10,000 steps only lowered the final loss to 0.159, yet the samples improved markedly (Section 6.3), so the loss value alone understates the progress in sample quality. The samples at step 1,000 already show recognisable but blurry cat faces, centred and with visible ears and eye positions, and a grey/blue colour cast.

6.2 FID during training

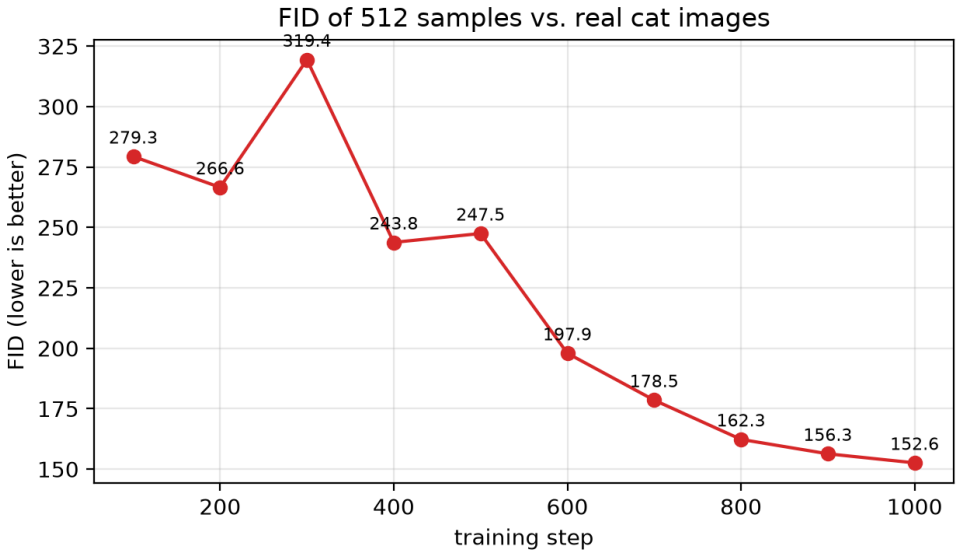


Figure 3: FID between 512 generated samples and the 500 real cat images in `data/val/cat`, every 100 steps.

step	100	200	300	400	500	600	700	800	900	1000
FID	279.3	266.6	319.4	243.8	247.5	197.9	178.5	162.3	156.3	**152.6**

Table 2: FID every 100 training steps (run `p1sjl5gm`).

FID compares the mean and covariance of Inception-v3 features of generated and real images; lower is better. Over 1,000 steps FID falls from 279.3 to **152.6**, a 45% reduction, but the path is not monotonic and has two distinct phases:

- **Steps 100–500: noisy, little net progress.** FID moves between 244 and 319, with a clear spike at step 300 (319.4, worse than at step 100). At this stage the samples are still colour blobs with a rough face layout, and each FID value comes from a single snapshot of rapidly changing weights (learning rate 10^{-3}, no weight averaging), so small changes in the network can shift the colour statistics of all 512 samples at once.
- **Steps 500–1,000: steady improvement.** FID drops monotonically, with the largest single gain between steps 500 and 600 (-49.6), and then smaller gains (-19.4, -16.2, -6.0, -3.7). The shrinking improvements show that progress is slowing, but FID is still falling at step 1,000, so the model has not converged.

The comparison with the loss curve is instructive: the loss has essentially flattened by step $\sim$200, whereas FID only starts improving consistently after step 500. A small further reduction in average noise-prediction error therefore still translates into clearly better samples, and the training loss alone is a poor indicator of sample quality. This matches Section 6.3, where 10,000 steps lowered the loss only from 0.182 to 0.159 yet produced much better images.

Absolute values should be read with care: 512 samples is a small set for estimating a 2048-dimensional covariance, the images are upsampled from 32×32 to Inception's 299×299 input, and the reference set has only 500 images, so FID here is mainly useful for the *trend*. Using a cat-only reference set (fix 3 above) matters: comparing cats against a cat/dog/wild mixture would keep FID high even for a perfect cat generator.

6.3 Samples after 10,000 steps



Figure 4: Last sample batch (32 images) after 10,000 training steps.

Compared with 1,000 steps the improvement is substantial. Faces are sharper, with distinct eyes, noses and pointed ears, and the cat is clearly separated from the background. The grey/blue cast is gone and colours look natural. The batch is also diverse: orange and grey tabbies, black-and-white "tuxedo" cats, white cats and Siamese-like faces all appear, so there is no sign of mode collapse. Remaining flaws are soft fur texture and occasionally asymmetric or smudged eyes, which is expected for a small U-Net (base width 16), only $T = 50$ steps and 10k iterations at 32×32.

6.4 Forward diffusion

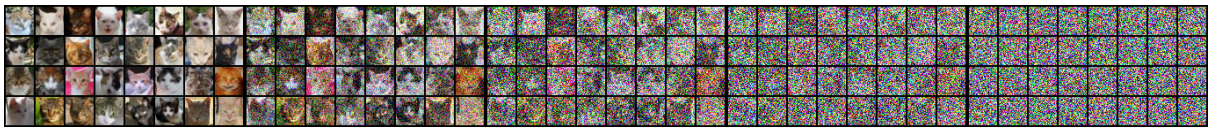


Figure 5: Forward process on the first training batch at $t = 0, 12, 25, 37, 49$ (0%, 25%, 50%, 75%, 99% of $T = 50$), left to right.

The forward process needs no learning: $\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t}\boldsymbol{\epsilon}$. With our cosine schedule the signal and noise scales at the five shown steps are:

t	0	12	25	37	49
signal $\sqrt{\bar{\alpha}_t}$	0.999	0.914	0.680	0.365	0.001
noise $\sqrt{1 - \bar{\alpha}_t}$	0.042	0.405	0.733	0.931	1.000

This matches the figure: at $t = 0$ the cats are unchanged; at $t = 12$ they are clearly visible under grain; at $t = 25$ signal and noise are of similar size and only the coarse layout (face position, dark vs. light fur) survives; at $t = 37$ the image is mostly noise; and at $t = 49$ it is indistinguishable from pure Gaussian noise. The cosine schedule removes information gradually rather than destroying most of it in the first few steps, which is why the middle panels still contain recognisable structure.

6.5 Backward diffusion

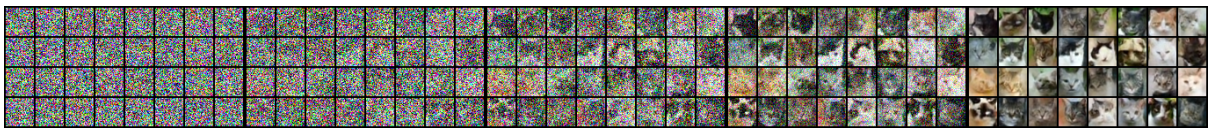


Figure 6: Reverse process of the 10k-step model, started from the $t = 49$ forward images, shown at $t = 49, 37, 25, 12, 0$, left to right.

Starting from the noisy $\mathbf{x}_{49}$ of the forward process, the model removes noise one step at a time. Reading the figure left to right:

- $t = 49$ and $t = 37$: both panels still look like pure noise; the first reverse steps only adjust low-frequency colour and brightness, which is hidden under the large remaining noise ($\sqrt{1 - \bar{\alpha}_{37}} \approx 0.93$).
- $t = 25$: faint cat silhouettes start to appear under the noise (dark and light regions where the head and ears will be).
- $t = 12$: the cats are clearly recognisable, with face layout, ears and fur colours decided, but still covered in grain.
- $t = 0$: clean, sharp cat faces with natural and diverse colours.

Two observations. First, the backward panels look like the forward panels in mirror order: at each shown t, the reverse image has about the same visible noise level as the forward image at that t. This is what a well-trained model should do, since the reverse chain is trained so that its intermediate $\mathbf{x}_t$ follow the same marginals $q(\mathbf{x}_t)$ as the forward chain. Coarse structure is decided in the middle of the chain and fine detail in the last steps. Second, the final images are *new* cats, not reconstructions of the original batch shown in Section 6.4 (compare the first rows of the two figures): at $t = 49$ we have $\sqrt{\bar{\alpha}_{49}} \approx 0.001$, so essentially no information about the original images is left, and the reverse process samples fresh images from the learned distribution.

Reproducibility

All commands are in `README.md`. In short:

```
python main.py --train_steps 1000 --save_and_sample_every 100 \
    --save_folder ./results/run_1k
python main.py --train_steps 1000 --save_and_sample_every 100 --fid \
    --save_folder ./results/run_1k_fid
python main.py --train_steps 10000 --save_and_sample_every 1000 \
    --save_folder ./results/run_10k
python main.py --visualize --load_path ./results/run_10k/model.pt \
    --save_folder ./results/run_10k_vis
python plot_curves.py --loss_run 7s0eri6w --fid_run <FID_RUN_ID>
```

(On macOS add `--dataloader_workers 0`.)